PURDUE UNIVERSITY | FORT WAYNE

WEEKLY BUILD ACTIVITY JOURNAL

CS 590 Low Code No Code for Rapid Prototyping

Student Name: Kishore Vasudevan
Week: 6
Date Range: 09/28/2026 - 10/04/2026
Course Work and/or Project: Week 6 In-Class Activity (Tech Signal News Aggregator) + Sprint 2 (GitHub Issue Triage Automation - In-Class Part 1 to Part 3 Step 8)

1. Weekly Focus and Goals

- **Goal 1:** Build and verify the self-hosted n8n RSS news aggregator workflow ("Tech Signal"), map normalized fields, and upsert items into the tech_signal_items Data Table with duplicate safety.
- **Goal 2:** Setup public GitHub repository (n8n-sprint2-issue-triage) and create two persistent n8n Data Tables (github_issue_ledger and github_issue_route_log) for Sprint 2.
- **Goal 3:** Build Sprint 2 event-driven webhook ingestion: local Webhook trigger node, event action gating (Opened action?), payload normalization (Normalize issue), and deterministic title classification (Classify title up to Part 3 Step 8).

2. Completed Work and Evidence

Completed work	Evidence	Why it matters to this week's objective
Week 6 Activity: n8n News Aggregator ("Tech Signal")	Exported news_aggregator.json, README.md with reflection, and populated tech_signal_items Data Table (7 columns).	Demonstrates core low-code retrieval, filtering, field normalization, and idempotent data table upsert in n8n.
Sprint 2 Part 1 & 2: GitHub Repo & Data Tables Setup	Public GitHub repo	Establishes the event source and persistent storage layers

PURDUE UNIVERSITY | FORT WAYNE

Completed work	Evidence	Why it matters to this week's objective
		required for event-driven triage and routing logs.
Sprint 2 Part 2 (Steps 5–7): Webhook Intake & Normalization	n8n workflow nodes: GitHub issue webhook (POST sprint2-issue-triage), Opened action? (IF node), and Normalize issue (Edit Fields node).	Safely receives GitHub webhooks, filters out non-opened actions, and normalizes nested event properties into a stable issue_key and schema.
Sprint 2 Part 3 (Step 8): Deterministic Title Classification	Classify title Switch node with 4 regex rules (\b(bug|error)\b, \b(feature|request)\b, \b(help|question)\b, fallback) connected to 4 route-setting nodes (Set bug route, Set feature route, Set help route, Set other route).	Replaces unpredictable LLM classification with deterministic, explainable regex matching for automated issue triage.

PURDUE UNIVERSITY | FORT WAYNE

3. Work in Progress and Next Step [1 point]

Work in progress	Current state	Next concrete action
Sprint 2 GitHub Issue Triage Workflow (Steps 9–11)	Completed up to Part 3 - Step 8 (title classification and route setting).	Implement the duplicate guard node (checking github_issue_ledger), configure ledger/route-log write nodes, start gh webhook CLI forwarding, and test live GitHub issues.

4. What You Learned

Core Learning & Outcome: learned the architectural difference between pull-based data ingestion (polling an RSS feed on a schedule) and push-based event-driven automation (listening for real-time GitHub Webhooks). I also learned how to implement deterministic classification

using regular expressions (\b(bug|error)\b) instead of relying on non-deterministic LLM calls, ensuring 100% predictable routing.

Prototype Dimension Tested: Implementation (configuring webhook payloads, regex matching, and n8n Data Table schema relationships) and Data Shape (normalizing raw nested JSON payloads into consistent record keys).

Unanswered Question: How does n8n handle high-concurrency webhook bursts when multiple GitHub issues are opened simultaneously—does it queue executions sequentially or require a dedicated external queue (like Redis/RabbitMQ) at production scale?

5. Challenge and Diagnosis [2 points]

What happened? During RSS feed mapping and Webhook payload normalization, property names varied significantly across feeds and events (e.g., isoDate vs pubDate in RSS, and nested body.action vs top-level properties in Webhooks).

What did you try or rule out? ruled out hardcoding single property names. I inspected the raw execution output JSON in n8n's input panel to identify the actual payload structure.

What will you try next / How was it resolved? For RSS, I used nullish coalescing expressions ({{ $json.isoDate ?? $json.pubDate }}). For GitHub webhooks, I mapped the nested payload path ({{ $json.body.action }}) and converted titles to lowercase (title_lower) before regex classification to ensure case-insensitive matching**.**

6. Response Plan [1 point]

Next Steps: Complete Sprint 2 Steps 9–11 in the next session: connect the duplicate check against github_issue_ledger, write route logs to github_issue_route_log, run gh webhook forward from the terminal, test issue redelivery, and final submission evidence.

Course Evidence Check

- I included evidence that a reviewer can inspect.
- I used only synthetic or course- approved data and did not include secrets, tokens, or personal information.

PURDUE UNIVERSITY | FORT WAYNE

- I named a real outcome and a real challenge rather than writing a generic status update.
- If this work has a cost, security, privacy, or extensibility implication, I named it in the relevant section above

DEPARTMENT OF COMPUTER SCIENCE

Engineering, Technology, and Computer Science Building

2101 Coliseum Blvd E. Fort Wayne, IN 46805- 1499